Abonnez-vous à DeepL Pro pour traduire des fichiers plus volumineux.
Visitez www.DeepL.com/pro pour en savoir plus.



Entrez dans la Matrice

Introduction à l'algèbre linéaire

Tristan Duquesne, Luc Lenôtre

Résumé : Vecteurs et matrices, en gros Version : 1.2

Sommaire

I Avant-propos 3
I.1 Règles générales 5

II Introduction 6
II.1 Alphabet grec 6
II.2 Cartes graphiques 7
II.3 Multiplication-accumulation fusionnée 8
II.4 Espace vectoriel 8
II.5 Applications linéaires 10
II.6 Composition 13

III Contraintes de code 14

IV Exercice 00 - Ajouter, soustraire et mettre à l'échelle 16
IV.0.1 Objectif 16
IV.0.2 Instructions 16

V Exercice 01 - Combinaison linéaire 19
V.0.1 Objectif 19
V.0.2 Instructions 19

VI Exercice 02 - Interpolation linéaire 21
VI.0.1 Objectif 21
VI.0.2 Instructions 21

VII Exercice 03 - Produit scalaire 23
VII.0.1 Objectif 23
VII.0.2 Instructions 23
VII.1 Produit scalaire dans les espaces vectoriels complexes 24

VIII Exercice 04 - Norme 25
VIII.0.1 Objectif 25
VIII.0.2 Instructions 25
VIII.1 Inégalité triangulaire 26

IX Exercice 05 - Cosinus 27
IX.0.1 Objectif 27
IX.0.2 Instructions 27
IX.1 Coefficient de corrélation de Pearson 29

X Exercice 06 - Produit vectoriel 31
X.0.1 Objectif 31

X.0.2 Instructions 31
X.1 Produit en coin et algèbre extérieure 32

XI Exercice 07 - Application linéaire, multiplication de matrices 34
XI.0.1 Objectif 34
XI.0.2 Instructions 34
XI.1 Matrices spéciales 36

XII Exercice 08 - Trait 38
XII.0.1 Objectif 38
XII.0.2 Instructions 38

XIII Exercice 09 - Transposition 40
XIII.0.1 Objectif 40
XIII.0.2 Instructions 40
XIII.1 Transposée conjuguée 41

XIV Interlude 00 - Résolution de systèmes d'équations linéaires 42

XV Exercice 10 - Forme échelonnée par lignes 44
XV.0.1 Objectif 44
XV.0.2 Instructions 44

XVI Exercice 11 - Déterminant 46
XVI.0.1 Objectif 46
XVI.0.2 Instructions 46
XVI.1 Le déterminant et l'algèbre extérieure 48

XVII Exercice 12 - Inverse 49
XVII.0.1 Goal 49
XVII.0.2 Instructions 49
XVII.1 Pseudo-inverse de Moore-Penrose 51

XVIII Interlude 01 - Théorème de rang-nullité 52
XVIII.0.1 Mais attendez, qu'est-ce qu'une image ? Qu'est-ce qu'un noyau ? 52

XIX Exercice 13 - Rang 54
XIX.0.1 Objectif 54
XIX.0.2 Instructions 54

XX Exercice 14 - Bonus : Matrice de projection 56
XX.0.1 Objectif 56
XX.0.2 Instructions 56

XXI Exercice 15 - Bonus : Espaces vectoriels complexes 60
XXI.0.1 Objectif 60

XXII Soumission et évaluation par les pairs 62

Chapitre I Avant-

propos

L'algèbre linéaire est un sujet très important que l'on retrouve presque partout en mathématiques, en informatique et en physique.

Elle consiste en l'étude des espaces vectoriels, qui sont constitués d'objets appelés vecteurs. Les transformations de vecteurs, appelées applications linéaires, sont généralement représentées par des objets appelés « matrices » dans le cas le plus courant (à dimension finie). Il existe également d'autres types d'espaces vectoriels, plus complexes, tels que les polynômes réels ou complexes, et les espaces de fonctions. L'algèbre linéaire nous permet d'étudier ces divers sujets dans un cadre unificateur. Cela rend les cas les plus compliqués plus faciles à comprendre, car leurs propriétés sont généralement les mêmes que celles des cas simples ; vous pouvez donc vous servir des cas simples pour visualiser mentalement ce qui se passerait dans les cas compliqués. C'est pourquoi nous nous concentrerons sur le cas fondamental des vecteurs et des matrices de dimension finie, en mettant l'accent sur les dimensions 2D et 3D, qui peuvent être facilement visualisées.

Quels sont donc quelques exemples d'application de l'algèbre linéaire ?

La mécanique newtonienne (tout ce qui concerne le mouvement à l'échelle humaine) regorge de vecteurs. De plus, toutes les formes « plus complexes » de la physique reposent généralement sur l'algèbre linéaire ou ses extensions (calcul à plusieurs variables, topologie, géométrie différentielle, algèbre des tenseurs, théorie des opérateurs…). On pourrait décrire la mécanique quantique comme l'algèbre linéaire et le calcul à plusieurs variables sur les nombres complexes. La relativité générale n'est qu'une application sophistiquée de la géométrie différentielle et de l'algèbre des tenseurs. Même la théorie des signaux peut avoir un « côté » algèbre linéaire, puisque les signaux ne sont au fond que des fonctions, et que les espaces de fonctions numériques ne sont qu'un cas particulier d'espaces vectoriels.

De plus, les jeux vidéo ne sont au fond que des simulations physiques ludiques : ils font donc largement appel à l'algèbre linéaire ! Vous voulez que Mario saute ? Vous aurez soit besoin d'un « vecteur d'impulsion de saut » pour le propulser vers le haut, et d'un vecteur « gravité » pour le ramener vers le bas ; soit d'un polynôme du deuxième degré pour définir la courbe du saut. Vous voulez créer des images de synthèse dignes du cinéma ? Le ray tracing n'est qu'une simulation du fonctionnement de la lumière, et il s'agit simplement de vecteurs de rebond appelés « rayons ». Tous ces objets sont principalement définis par l'algèbre linéaire. De plus, les termes « 2D » et « 3D » (c'est-à-dire la notion de « dimension ») trouvent leur origine dans l'algèbre linéaire. L'infographie regorge de vecteurs et de leurs transformations par le biais de matrices. Vous souhaitez faire pivoter un sprite 2D pour un sort de feu ? C'est une matrice. Vous souhaitez

créer une caméra 3D pour un FPS ? C'est une matrice. Le GPU a été inventé essentiellement pour bien gérer les opérations d'algèbre linéaire (voir ci-dessous). Le calcul haute performance et l'analyse numérique dépendent donc fortement de l'algèbre linéaire.

Les statistiques (et donc la science des données et l'apprentissage automatique) reposent également en grande partie sur l'algèbre linéaire. Les données numériques sont souvent représentées sous forme de vecteurs (un « nuage de points ») dans des espaces vectoriels de grande dimension. Il existe des analogies importantes à établir entre les concepts statistiques (respectivement : covariance, variance, écart-type, coefficient de corrélation de Pearson, normalisation d'une courbe gaussienne) et les opérations d'algèbre linéaire (respectivement : produit scalaire, norme quadratique, norme, cosinus de l'angle entre deux vecteurs, normalisation d'un vecteur). Comprendre la géométrie de l'algèbre linéaire est donc très utile pour appréhender la géométrie sous-jacente aux statistiques.

Il existe de très nombreuses autres applications de l'algèbre linéaire et de ses variantes (dans des domaines allant de la cryptographie à l'écologie), mais celles que nous avons énumérées ici constituent les cas d'utilisation les plus fréquents dans la pratique. Une fois que vous l'aurez apprise, vous la verrez de plus en plus souvent autour de vous.

En raison de son importance, du peu de prérequis nécessaires à son apprentissage (hormis l'arithmétique de base) et de tous les domaines des mathématiques et des sciences auxquels elle ouvre la voie, l'algèbre linéaire a été choisie comme point d'entrée standard vers les mathématiques avancées, partout dans le monde. Nous espérons que vous apprécierez autant que nous son utilité dans la pratique !

I.1 Règles générales

- Pour ce module, les prototypes de fonctions sont décrits en Rust, mais vous pouvez utiliser le langage de votre choix. Il existe toutefois quelques contraintes concernant le langage que vous choisirez :
    - Il doit prendre en charge les types génériques
    - Il doit prendre en charge les fonctions en tant qu'éléments de première classe (par exemple : prendre en charge les expressions lambda)
    - Facultatif : prise en charge de la surcharge d'opérateurs
- Nous vous recommandons d'utiliser du papier si nécessaire. Dessiner des modèles visuels, prendre des notes sur le vocabulaire technique… De cette manière, il vous sera plus facile de vous familiariser avec les nouveaux concepts que vous découvrirez, et de les utiliser pour approfondir votre compréhension de concepts de plus en plus liés. De plus, effectuer des calculs de tête est très difficile et source d'erreurs ; le fait d'avoir noté toutes les étapes vous aidera à repérer où vous vous êtes trompé le cas échéant.
- Ne restez pas bloqué parce que vous ne savez pas comment résoudre un exercice : profitez de l'apprentissage entre pairs ! Vous pouvez demander de l'aide sur Slack (42 ou 42-AI) ou Discord (42-AI), par exemple.
- Vous n'êtes pas autorisé à utiliser de bibliothèque mathématique, même celle incluse dans la bibliothèque standard de votre langage, sauf indication contraire explicite.
- Pour chaque exercice, vous devrez peut-être respecter une complexité temporelle et/ou spatiale donnée. Celles-ci seront vérifiées lors de la révision par les pairs.
    - La complexité temporelle est calculée par rapport au nombre d'instructions exécutées.
    - La complexité en espace est calculée par rapport à la quantité maximale de mémoire allouée simultanément.
    - Ces deux complexités doivent être calculées par rapport à la taille de l'entrée de la fonction (un nombre, la longueur d'une chaîne de caractères, etc.).

Chapitre II

Introduction

Pour ce module, nous vous recommandons **vivement** la série de vidéos « Essence of linear algebra » de 3blue1brown, qui propose une explication très claire, complète et visuelle de ce sujet.

Ces vidéos sont tellement bien faites que vous aurez sans doute envie de les regarder une première fois avant de vous lancer dans ce module ; vous y reviendrez pour revoir certaines parties au cours du module, et vous aurez envie de les revoir une nouvelle fois après ce module, peut-être quelques mois plus tard. Nous ne plaisantons pas.

Beaucoup de personnes à qui nous avions recommandé ces vidéos avant qu'elles ne se lancent dans la programmation de la physique des jeux vidéo, la science des données ou l'infographie ont décidé de ne pas les regarder avant bien plus tard. La réaction a été UNANIME : celles qui avaient regardé les vidéos n'avaient que des éloges à faire sur leur incroyable utilité ; celles qui n'avaient regardé les vidéos que plus tard dans leur apprentissage de l'algèbre linéaire regrettaient profondément le temps qu'elles auraient pu économiser si elles avaient regardé les vidéos dès le début.

Ne perdez pas votre temps. REGARDEZ. CES. VIDÉOS. Même si vous ne comprenez pas tout au début, la présentation structurée et les illustrations géométriques vous fourniront une bonne base contextuelle et des modèles mentaux qui s'avéreront essentiels à mesure que vous apprendrez les différents aspects de l'algèbre linéaire.

II.1 L'alphabet grec

Comme vous le savez peut-être, les mathématiques (et la physique) utilisent beaucoup l'alphabet grec. Si vous souhaitez l'apprendre, le voici :

Minuscules	Majuscules	Nom
α	A	Alpha
β	B	Bêta
γ	Γ	Gamma
δ	Δ	Delta
ϵ	E	Epsilon
ζ	Z	Zêta
η	E	Eta
θ	Θ	Thêta
ι	I	Iota
κ	K	Kappa
λ	Λ	Lambda
μ	M	Mu
ν	N	Nu
ξ	Ξ	Xi
o	O	Omicron
π	Π	Pi
ρ	R	Rho
σ	Σ	Sigma
τ	T	Tau
υ	Υ	Upsilon
ϕ	Φ	Phi
χ	X	Chi
ψ	Ψ	Psi
ω	Ω	Oméga

II.2 Unités de traitement graphique

Un processeur graphique (GPU) est un composant matériel présent dans de nombreux ordinateurs modernes et supercalculateurs, qui permet d'effectuer des calculs parallèles. L'architecture de traitement d'un GPU est dite SIMD (pour « Single Instruction, Multiple Data », soit « instruction unique, données multiples ») : cela signifie qu'un GPU fonctionne bien lorsqu'il peut exécuter le même code (c'est-à-dire des fonctions sans branchements conditionnels divergents ni instructions « goto ») simultanément sur différentes valeurs d'entrée du même type (généralement des nombres à virgule flottante). On peut comparer cela à une usine où une série de machines identiques effectue la même opération sur des objets, simultanément, sur plusieurs lignes de production parallèles.

Comme leur nom l'indique, les GPU étaient à l'origine destinés aux calculs graphiques, qui reposent largement sur l'exécution d'opérations d'algèbre linéaire. En raison de l'omniprésence de l'algèbre linéaire en mathématiques, l'utilisation des GPU s'est étendue au-delà des jeux vidéo et de l'imagerie de synthèse, et la plupart des supercalculateurs actuels s'appuient sur le calcul par GPU.

De nombreux processeurs modernes s'inspirent également du modèle de calcul SIMD et prennent en charge des opérations SIMD spécifiques.

Les opérations d'algèbre linéaire consistent principalement à : additionner des vecteurs (une addition par coordonnée

de deux vecteurs), la mise à l'échelle de vecteurs (multiplication de toutes les coordonnées d'un vecteur donné par le même nombre réel ou complexe) et les multiplications de matrices. Les multiplications de matrices ne sont qu'une succession répétée d'une opération appelée **produit scalaire** (ou **produit interne**) de deux vecteurs. Toutes ces opérations sont parfaitement adaptées au SIMD, puisqu'elles ne sont qu'une succession d'additions ou de multiplications sur des tableaux de données (qui représentent nos vecteurs). Codifier ces différentes opérations pour mieux les comprendre est un exercice fondamental. C'est pourquoi vous aurez l'occasion de toutes les coder au cours de ce module.

II.3 Multiplication-accumulation fusionnée

Sous l'architecture x86, il existe des instructions appelées : `vfmadd132ps`, `vfmadd213ps` et `vfmadd231ps` (bienvenue dans l'univers x86 :D), qui nous permettent de calculer ce qu'on appelle l'opération « **multiplication-accumulation fusionnée** ».

Vérifiez ce que font ces instructions, et vous vous rendrez compte qu'elles pourraient s'avérer utiles.
;)

La bibliothèque standard du langage C (ainsi que celle de nombreux autres langages) dispose une fonction permettant d'effectuer cette opération (mais nous ne vous dirons pas laquelle, alors cherchez-la vous-même !).

II.4 Espace vectoriel

Ce qui suit, à savoir une liste concise mais succincte d'informations théoriques sur les espaces vectoriels, pourra sans doute vous être utile à consulter et à relire au fur et à mesure que vous avancerez dans ce module. Il est utile de travailler à la fois sur les applications pratiques et sur la théorie, afin de développer progressivement une compréhension plus approfondie. Ne vous découragez pas si vous ne comprenez pas tout immédiatement : c'est tout à fait normal lorsqu'on aborde un nouvel écosystème d'informations.

Si vous vous souvenez du concept de « groupes (algébriques) » vu dans le module sur l'algèbre booléenne, cette section vous paraîtra beaucoup plus facile à comprendre (si vous n'avez pas encore suivi ce module, nous vous recommandons de le faire, car les ensembles, la logique et les structures algébriques sont encore plus fondamentaux que l'algèbre linéaire).

Comprendre les choses du point de vue des structures algébriques peut sembler difficile au premier abord, car le vocabulaire est riche. Cependant, ce vocabulaire est extrêmement utile, car il transforme les propriétés mathématiques de divers ensembles en quelque chose de simple, comme connaître les règles d'un jeu de cartes. Ce que vous avez le droit de faire, et ce qui est interdit. Si vous êtes capable de vous souvenir des règles de 4 ou 5 jeux de société ou jeux de cartes, vous avez sans aucun doute ce qu'il faut pour apprendre le langage des structures algébriques (souvent appelé « algèbre abstraite », un nom trompeur qui fait peur).

En réalité, ce n'est vraiment pas si compliqué à apprendre, car il s'agit essentiellement UNIQUEMENT d'apprendre du vocabulaire, et non de maîtriser des méthodes algorithmiques complexes. C'est pourquoi nous avons jugé utile d'inclure ce qui suit : voici le

définition d'un espace vectoriel, le principal type de structure étudié en algèbre linéaire, du point de vue des « structures algébriques » (c'est-à-dire les règles pouvant être appliquées à un ensemble donné afin de transformer cet ensemble en « espace vectoriel »).

Un espace vectoriel V est une structure qui associe :

- Un **corps** K (en gros : une structure algébrique dans laquelle on peut additionner, soustraire, multiplier et diviser des éléments selon les règles de l'arithmétique habituelle). Les éléments de K sont appelés « scalaires ». K est généralement choisi parmi les nombres réels ou les nombres complexes. Les nombres réels sont généralement représentés sous forme de nombres à virgule flottante ; les nombres complexes sous forme de paires de nombres à virgule flottante, avec des opérations de multiplication et de division spécifiques. On désigne généralement les scalaires par des lettres grecques.
- Un **groupe commutatif** V (en gros : une structure algébrique dans laquelle on peut additionner et soustraire des éléments selon les règles de l'arithmétique habituelle, mais pas nécessairement les multiplier ou les diviser). Les éléments de V sont appelés « vecteurs ». Pour des dimensions finies, V est toujours équivalent à K^n, où n est un nombre naturel, et n est appelé la « dimension » de V. Cela signifie que, dans des dimensions finies, V peut TOUJOURS être considéré comme une liste, un tableau ou un tuple contenant n éléments de K. Cela signifie également que tout corps K peut être considéré comme un espace vectoriel unidimensionnel. On désigne généralement les vecteurs par des lettres latines.
- Une opération, appelée multiplication scalaire (ou multiplication par un scalaire), permet de combiner des éléments de K et *de* V. On ne peut pas additionner ou soustraire un élément de K à un élément de V, mais on peut multiplier un élément de K par un élément de V, ce qui donne un élément de V.

L'opération de multiplication scalaire doit également satisfaire aux propriétés suivantes :

- La multiplication scalaire prend deux éléments, l'un dans K, l'autre dans V, et renvoie un élément de V :

$$\lambda \in \mathrm{K},\ \forall u \in V,\ \lambda u \in V$$

- Pseudo-distributivité sur les vecteurs (voir « distributivité ») :

$$\forall \lambda \in \mathrm{K},\ \forall (u, v) \in V^2,\ \lambda(u + v) = \lambda u + \lambda v$$

- Pseudo-distributivité sur les scalaires (voir « distributivité ») :

$$\forall (\lambda, \mu) \in \mathrm{K}^2,\ \forall u \in V,\ (\lambda + \mu)u = \lambda u + \mu u$$

- Pseudo-associativité multiplicative de la multiplication scalaire (voir « associativité ») :

$$\forall (\lambda, \mu) \in \mathrm{K}^2,\ \forall u \in V,\ (\lambda\mu)u = \lambda(\mu u)$$

- Compatibilité de l'identité multiplicative du corps avec la multiplication scalaire (voir « élément identitaire ») :

$$\forall u \in V,\ 1_K\, u = u$$

Pour une liste complète et détaillée des axiomes/propriétés (c'est-à-dire les « règles du jeu ») des espaces vectoriels, consultez la page Wikipédia consacrée aux espaces vectoriels

Si les structures algébriques (qui sont EXTRÊMEMENT utiles pour comprendre les mathématiques avancées) ou les nombres complexes vous intéressent, vous pouvez consulter les PDF suivants, rédigés par l'un des auteurs de ce module (ces PDF sont également disponibles en français).

L'algèbre linéaire étudie également un type particulier d'espace vectoriel, appelé « espace euclidien » (également appelé « espace à produit scalaire » ou IPS), auquel on ajoute un autre opérateur : le fameux **produit scalaire**. Cet opérateur prend deux vecteurs et renvoie un scalaire. De nombreux autres objets d'étude en physique et dans d'autres domaines qui s'appuient sur l'algèbre linéaire ne sont que des « espaces vectoriels dotés d'une structure supplémentaire » (par exemple : espaces vectoriels topologiques, espaces métriques, variétés, champs vectoriels, K-algèbres, algèbres de Lie, algèbres de tenseurs, algèbres géométriques, espaces de fonctions de vecteurs aléatoires...)

II.5 Applications linéaires

Soient V et W deux espaces vectoriels sur le même corps K.

Soient $u, v \in V$ deux vecteurs et $\lambda \in K$ un scalaire (où K est le corps associé à l'espace vectoriel V).

La fonction $f : V \to W$ est dite **application linéaire** (ou **transformation linéaire**) si :

$$\forall (u, v) \in V^2,\ f(u + v) = f(u) + f(v)$$ (ce qu'on appelle aussi « l'additivité »)

$$\forall \lambda \in K,\ \forall u \in V,\ f(\lambda u) = \lambda f(u)$$ (ce qu'on appelle l'« homogénéité de degré 1 »)

Si ces propriétés sont vérifiées, on dit que f associe **linéairement** des valeurs de l'espace vectoriel V à un nouvel espace vectoriel W. Géométriquement, « linéairement » signifie que deux droites quelconques qui sont parallèles dans l'espace d'origine restent parallèles après la transformation (ceux qui ont regardé 3blue1brown devraient le savoir à présent).

Ce sont là des conditions nécessaires et suffisantes : pour prouver que f est linéaire, il faut démontrer que les deux équations ci-dessus sont vraies. Voir : Nécessité et suffisance. Notons que la deuxième équation implique que le vecteur nul 0_V de l'espace d'entrée V (le vecteur dont tous les coefficients sont nuls) est projeté sur le vecteur nul 0_W de l'espace de sortie W.

Vous pouvez également comprendre ce qui se passe à l'aide des diagrammes suivants (appelés « **diagrammes commutatifs** » en théorie des catégories) :

$$\begin{array}{ccc} (u, v) \in V \times V & \xrightarrow{(f, f)} & (f(u), f(v)) \in W \times W \\ \downarrow{\scriptstyle +_V} & & \downarrow{\scriptstyle +_W} \\ u + v \in V & \xrightarrow{f} & f(u+v) = f(u) + f(v) \in W \end{array}$$

$$\begin{array}{ccc} (\lambda, u) \in K \times V & \xrightarrow{(id_K, f)} & (\lambda, f(u)) \in K \times W \\ \downarrow{\scriptstyle *_V} & & \downarrow{\scriptstyle *_W} \\ \lambda u \in V & \xrightarrow{f} & f(\lambda u) = \lambda f(u) = W \end{array}$$

ou, sous une forme plus concise (mais moins explicite) :

$$\begin{array}{ccc} V \times V & \xrightarrow{(f, f)} & W \times W \\ \downarrow{\scriptstyle +_V} & & \downarrow{\scriptstyle +_W} \\ V & \xrightarrow{f} & W \end{array}$$

$$\begin{array}{ccc} K \times V & \xrightarrow{(id_K, f)} & K \times W \\ \downarrow{\scriptstyle *_V} & & \downarrow{\scriptstyle *_W} \\ V & \xrightarrow{f} & W \end{array}$$

Ces diagrammes commutatifs, lorsqu'ils sont véridiques, signifient que vous pouvez emprunter n'importe quel chemin entre le point de départ (deux éléments de V) et le point d'arrivée (un seul élément de W), et vous obtiendrez le même résultat. Le choix d'« appliquer d'abord f » ou d'« utiliser l'opérateur de l'espace vectoriel » n'a pas d'importance, tant que l'autre opération est celle qui suit. Prenez le temps de réfléchir un instant pour comprendre en quoi ces diagrammes sont exactement équivalents aux équations de linéarité écrites ci-dessus. Cela vous aidera à analyser plus en détail ce qui se passe.

En algèbre linéaire, une application linéaire f correspond généralement à une matrice. Pour être précis, si V est de dimension n et W de dimension m, et si V et W sont chacun dotés d'une base (un système de coordonnées choisi), alors f peut être représentée de manière unique par une matrice $n \times m$, c'est-à-dire une matrice comportant n lignes et m colonnes.

L'ensemble des applications linéaires de V vers W est généralement noté $L(V, W)$, et l'espace des matrices de V vers W, pour des bases données B sur V et C sur W, est généralement noté $Mat_{B,C}(V, W)$, ou plus simplement, $K^{dim(W) \times dim(V)}$ (la dimension de l'espace de sortie est l'opérande de gauche !).

Ainsi, par exemple, l'espace des matrices 2 × 3 dont les coefficients sont des nombres réels (c'est-à-dire les matrices réelles 2 × 3) est $R^{2\times3}$, et correspond aux applications linéaires réelles de R^3vers R^2, c'est-à-dire $L(R^3, R^2)$.

Il est très important de noter que les espaces de matrices sont également des espaces vectoriels, puisqu'ils satisfont aux axiomes des espaces vectoriels (on peut multiplier une matrice par un scalaire, ainsi qu'additionner des matrices de même forme).

Voici un exemple de matrice carrée $A \in \mathrm{R}^{3\times4}$ (une matrice de réels de taille 3×4), représentant une application linéaire f qui prend en entrée un vecteur à 4 dimensions et renvoie en sortie un vecteur à 3 dimensions :

$$\begin{bmatrix} 8 & 2 & 3 & 3{,}73 \\ 7 & 5 & 0 & -1 \\ 17 & -32{,}3 & -4 & 5 \end{bmatrix}$$

Une matrice, considérée comme une transformation linéaire, peut être utilisée comme suit :

$$f(u) = v \Leftrightarrow Au = v$$

Ce qui correspond à une simple multiplication matricielle (que vous allez

mettre en pratique ci-dessous). Ainsi :

$A(u + v) = Au + Av$, c'est-à-dire que la distributivité de la multiplication matricielle

est l'additivité ; et

$A(\lambda u) = \lambda(Au) = (\lambda A)u$, l'homogénéité de degré 1 signifie que les matrices commutent avec la multiplication scalaire des vecteurs.

La matrice identité (souvent notée I_n) est une matrice carrée de taille $n \times n$ qui, lorsqu'elle est multipliée par un vecteur, n'a aucun effet sur ce vecteur.

La matrice identité $I_n \in R^{n\times n}$ est définie comme suit :

$$I_{ij} = \delta_{ij} = \delta^i_j$$

où δ est le **delta de Kronecker**. Cela signifie que la matrice identité comporte des 1 sur sa diagonale principale et des 0 partout ailleurs.

Pour une matrice A donnée, l'élément de la matrice situé à la ligne i et à la colonne j est noté
A_{ij} .

Ainsi, la matrice identité I_3 de dimension 3 est définie comme suit :

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Une fois que vous maîtrisez la multiplication des matrices, essayez de déterminer comment $\forall v \in$
R^3 , $I_3\, v = vI_3 = v$.

II.6 Composition

L'opérateur de composition est défini comme suit :

Soit $f : A \to B$ (f est une fonction définie de l'ensemble A vers l'ensemble B) et $g : B \to C$. Il existe alors une fonction $h : A \to C$ telle que :

$$\forall x \in A,\ h(x) = (g \circ f)(x) = g(f(x))$$

On lit généralement cela « g après f de x », ou « g de f de x ».

Pour simplifier, une composition de fonctions est une manière de désigner une succession spécifique de fonctions. Cette composition peut exister si et seulement si l'« espace intermédiaire » B correspond (c'est-à-dire que l'on peut appliquer g à la sortie de f si et seulement si le type de sortie de f est le même que le type d'entrée de g).

Lorsqu'on travaille avec des matrices, l'opérateur de composition pour les applications linéaires peut être simplement compris comme une succession de multiplications matricielles. Cela signifie que **la multiplication matricielle AB entre les matrices A et B fonctionne si et seulement si le nombre de colonnes de A correspond au nombre de lignes de B.**

Soient A, B, C trois matrices de $\mathrm{K}^{n\times n}$, représentant respectivement trois applications linéaires f, g, h de $L(\mathrm{K}^n)$ et soit u un vecteur de K^n. La composition des matrices s'écrit comme suit :

$$(f \circ g \circ h)(u) = A \times B \times C \times u = ABCu$$



La multiplication matricielle est associative, on peut donc omettre les parenthèses.
Cependant, pour des raisons d'efficacité de calcul, il est souvent préférable de déterminer s'il est plus rapide de calculer d'abord AB, puis $(AB)C$, plutôt que de calculer d'abord BC, puis $A(BC)$. L'idée est de réduire d'abord le « terme du milieu », qui est le plus grand, afin de réduire la charge de calcul de la deuxième étape.

Chapitre III

Contraintes de code

Pour ce module, vous devrez définir deux structures :

```
struct Vector::<K> {
    // ...
}

struct Matrix::<K> {
    // ...
}
```

Les structures ci-dessus sont déclarées en **Rust** et correspondent à un ensemble de données, sans méthode. N'hésitez pas à les adapter à la syntaxe la plus appropriée de votre langage (et à y ajouter vos méthodes si tel est le choix de conception que vous optez).

Nous vous recommandons d'implémenter certaines fonctions utilitaires, telles que :

- Une fonction renvoyant la taille et la forme d'un vecteur ou d'une matrice.
- Une fonction permettant de déterminer si une matrice est carrée.
- Une fonction permettant d'afficher un vecteur ou une matrice sur la sortie standard.
- Une fonction permettant de transformer un vecteur en matrice, et inversement.

Quelques points à noter.

Dans la suite, les prototypes des fonctions définiront des arguments génériques pour chacune d'entre elles. Cependant, vous n'avez à les implémenter que pour les nombres réels, qui seront définis comme étant de type `f32` ; c'est ce que vous devez comprendre lorsque vous voyez `K`. Notez toutefois qu'il y a un bonus à la fin de ce module : il s'agit de refaire tous les exercices en implémentant le corps K comme votre propre type de nombre `complexe`, plutôt que `comme f32`. Si vous avez travaillé de manière claire et générique, cela ne devrait pas nécessiter trop de modifications.

Certains des prototypes présentés en Rust relèvent davantage du « pseudocode » que du code proprement dit : une bonne bibliothèque d'algèbre linéaire en Rust s'appuierait sur le système de traits de Rust, et il faudrait donc spécifier dans les signatures les traits implémentés par vos génériques. De manière générale, il faut même s'attendre à ce que les types génériques fournis soient en réalité soumis à une forme de contrainte de type dans une implémentation réelle (par exemple, en créant un type `Field` et en faisant en sorte que le générique `K` étende `Field` ; c'est particulièrement le cas pour les langages fonctionnels). Il est donc probable que les exemples présentés dans certains exercices, tels quels, ne puissent pas être compilés.

De plus, dans la plupart des exercices de ce module, on peut généralement considérer que *V* est un espace vectoriel quelconque, de dimension finie, défini sur son corps scalaire K. Ainsi, *V*, en tant que générique, désigne un ensemble pouvant contenir des scalaires, des vecteurs, des matrices, des tenseurs, etc.

Enfin, de nombreuses fonctions des exercices stockent leur résultat dans l'une de leurs entrées. Il s'agit d'un choix. Ce type de fonction peut s'avérer beaucoup plus rapide lorsque vous utilisez votre objet comme accumulateur, mais vous préférerez peut-être que les entrées soient immuables (c'est-à-dire une « fonction pure ») et que la fonction renvoie simplement votre valeur. Si le langage de votre choix ne permet pas de telles constructions orientées objet, ou si vous préférez simplement la syntaxe des fonctions pures, vous êtes bien sûr autorisé à créer à la place une fonction qui renvoie un scalaire, un vecteur ou une matrice en sortie, tout en conservant ses entrées immuables. Vous pouvez également implémenter les deux versions, si vous le jugez utile : la fonction pure et la fonction accumulatrice. Nous demandons toutefois un certain degré de cohérence : si vous choisissez de n'implémenter qu'une seule version, respectez ce choix tout au long du module.

Conclusion : il reste donc une certaine marge de liberté d'interprétation à l'étudiant et à son évaluateur pour vérifier si la signature de la fonction est valide. Ce qui importe vraiment, c'est que la fonction mathématique sous-jacente soit implémentée de manière :

- générique, afin que l'ensemble de la bibliothèque de code écrite au cours de ce module puisse fonctionner avec d'autres corps que les nombres réels ;
- cohérente, afin que la bibliothèque présente un style et une utilisation homogènes ;
- fidèle, c'est-à-dire que la fonction implémentée soit une véritable implémentation du concept mathématique requis, et non autre chose.



Dans les exercices suivants, n, m et p correspondent chacun à une composante de dimension d'un vecteur ou d'une matrice. Cela signifie que $O(nm)$ pour une matrice *de n lignes sur m colonnes* signifie « pas plus d'opérations qu'un multiple constant du nombre de lignes multiplié par le nombre de colonnes ».



Sauf indication contraire, toutes les matrices présentées dans les exemples sont organisées selon **l'ordre des colonnes**.

Chapitre IV

Exercice 00 - Addition, soustraction et mise à l'échelle

	Exercice : 00
Addition, soustraction et mise à l'échelle	
Fonctions mathématiques autorisées : Aucune	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : $O(n)$	

IV.0.1 Objectif

Vous devez écrire des fonctions permettant d'additionner et de soustraire deux vecteurs ou deux matrices de même taille, ainsi qu'une fonction permettant de multiplier un vecteur ou une matrice par un scalaire (c'est-à-dire de « mettre à l'échelle »).

Vous devez également fournir une fonction principale permettant de tester vos fonctions, prête à être compilée (si nécessaire) et exécutée.

IV.0.2 Instructions

Vous devez écrire 3 fonctions.

- La première doit calculer l'addition de deux vecteurs.

- La deuxième doit calculer la soustraction d'un vecteur par un autre vecteur.

- La troisième doit calculer la mise à l'échelle d'un vecteur par un scalaire.

Si l'opération mathématique n'a pas de sens (par exemple, additionner un vecteur et un scalaire, ou

des vecteurs de tailles différentes), le résultat n'est pas défini.

Les prototypes des fonctions à écrire sont les suivants :

```
impl<K> Vector<K> {
    fn add(&mut self, v: &Vector<K>);
    fn sub(&mut self, v: &Vector<K>);
    fn scl(&mut self, a: K);
}

impl<K> Matrix<K> {
    fn add(&mut self, v: &Matrix<K>);
    fn sub(&mut self, v: &Matrix<K>);
    fn scl(&mut self, a: K);
}
```

Exemples :

```
let mut u = Vector::from([2., 3.]);
let v = Vector::from([5., 7.]);
u.add(v);
println!("{}", u);
// [7,0]
// [10.0]

let mut u = Vector::from([2., 3.]);
let v = Vector::from([5., 7.]);
u.sub(v);
println!("{}", u);
// [-3,0]
// [-4,0]

let mut u = Vector::from([2., 3.]);
u.scl(2.);
println!("{}", u);
// [4,0]
// [6,0]



let mut u = Matrix::from([
    [1., 2.],
    [3., 4.]
]);
let v = Matrix::from([
    [7., 4.],
    [-2., 2.]
]);
u.add(v);
```

```
println!("{}", u);
// [8,0 ; 6,0]
// [1,0, 6,0]

let mut u = Matrix::from([
    [1., 2.],
    [3., 4.]
]);
let v = Matrix::from([
    [7., 4.],
    [-2., 2.]
]);
u.sub(v);
println!("{}", u);
// [-6,0, -2,0]
// [5,0, 2,0]

let mut u = Matrix::from([
    [1., 2.],
    [3., 4.]
]);
u.scl(2.);
println!("{}", u);
// [2,0, 4,0]
// [6,0, 8,0]
```

Chapitre V

Exercice 01 - Combinaison linéaire

	Exercice : 01
Combinaison linéaire	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : $O(n)$	

V.0.1 Objectif

Vous devez écrire une fonction qui calcule une combinaison linéaire des vecteurs fournis, en utilisant les coefficients scalaires correspondants.

Vous devez également fournir une fonction principale permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.



`Le n dans la limite de complexité correspond à la dimension du vecteur/de la matrice, c'est-à-dire au nombre total de coordonnées.`

V.0.2 Instructions

- Soit $u = (u_1, ..., u_k) \in V^k$ une liste de taille k contenant des vecteurs (V est un espace vectoriel).
- Soit $\lambda = (\lambda_1, ..., \lambda_k) \in K^k$ une liste de taille k, contenant des scalaires.

Vous devez calculer la combinaison linéaire des vecteurs de u, pondérée par leurs

coefficients dans λ.

Si les deux tableaux fournis en entrée ne sont pas de même taille, ou si le contenu des tableaux est incohérent, le résultat n'est pas défini.

Le prototype de la fonction à écrire est le suivant :

```
fn linear_combination::<K>(u: &[Vector<K>], coefs: &[K]) -> Vector<K>;
```

Exemples :

```
let e1 = Vector::from([1., 0., 0.]);
let e2 = Vector::from([0., 1., 0.]);
let e3 = Vector::from([0., 0., 1.]);

let v1 = Vector::from([1., 2., 3.]);
let v2 = Vector::from([0., 10., -100.]);

println!("{}", linear_combination<Vector<f32>, f32>([e1, e2, e3], [10., -2.,
    0,5]));
// [10.]
// [-2.]
// [0,5]

println!("{}", linear_combination<Vector<f32>, f32>([v1, v2], [10., -2.]));
// [10.]
// [0.]
// [230.]
```

Chapitre VI

Exercice 02 - Interpolation linéaire

	Exercice : 02
Interpolation linéaire	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : $O(n)$	

VI.0.1 Objectif

Vous devez écrire une fonction qui calcule une interpolation linéaire entre deux objets de même type.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.



`Le n dans la limite de complexité correspond à la dimension du vecteur/de la matrice, c'est-à-dire au nombre total de coordonnées.`

VI.0.2 Instructions

- Soit $t \in [0\ ;\ 1] (\subset \mathrm{R})$ (c'est-à-dire que t est un nombre réel, et $0 \leq t \leq 1$) un scalaire.
- Soit $f : (V \times V \times [0\ ;\ 1]) \rightarrow V$ la fonction à implémenter.

La valeur de t nous permet de « glisser » entre ces deux valeurs.

- Si $t = 0$, alors $f(u, v, t) = u$.
- Si $t = 1$, alors $f(u, v, t) = v$.
- Si $t = 0{,}5$, alors la fonction renvoie une valeur située exactement à mi-chemin entre u et v (l'isobarycentre, que l'on peut considérer comme le centre de gravité des deux points).

Le prototype de la fonction à écrire est le suivant :

```
fn lerp::<V>(u: V, v: V, t: f32) -> V;
```

Exemples :

```
println!("{}", lerp(0., 1., 0.));
// 0,0

println!("{}", lerp(0., 1., 1.));
// 1,0

println!("{}", lerp(0., 1., 0,5));
// 0,5

println!("{}", lerp(21., 42., 0,3));
// 27,3

println!("{}", lerp(Vector::from([2., 1.]), Vector::from(4., 2.), 0,3));
// [2,6]
// [1,3]

println!("{}", lerp(Matrix::from([[2., 1.], [3., 4.]]), Matrix::from([[20.,
    10.], [30., 40.]]), 0.5));
// [[11., 5,5]
// [16,5, 22.]]
```



Même si l'interpolation linéaire est le plus souvent utilisée pour obtenir des points sur le segment compris entre u et v, techniquement, la formule devrait vous donner n'importe quel point sur la droite infinie tracée par les points u et v, si n'importe quelle valeur de t est acceptée. Cela est pertinent, par exemple, lors de la définition de rayons en lancer de rayons.

Chapitre VII

Exercice 03 - Produit scalaire

	Exercice : 03
Produit scalaire	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n)$	
Complexité en espace maximale : $O(n)$	

VII.0.1 Objectif

Vous devez écrire une fonction qui calcule le produit scalaire de deux vecteurs de même dimension.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

VII.0.2 Instructions

Soient $u, v \in V$, où V est un espace vectoriel de dimension finie sur les nombres réels R (représentés par le type `f32`).

La fonction doit calculer et renvoyer le scalaire $\langle u|v\rangle = u \cdot v$, appelé produit scalaire ou produit interne de u et v.

Si les deux vecteurs ont des dimensions différentes, le comportement est

indéfini. Le prototype de la fonction à écrire est le suivant :

```
impl<K> Vector::<K> {
    fn dot::<K>(&self, v: Vector::<K>) -> K;
}
```

Exemples :

```
let mut u = Vector::from([0., 0.]);
let v = Vector::from([1., 1.]);
println!("{}", u.dot(v));
// 0,0

let mut u = Vector::from([1., 1.]);
let v = Vector::from([1., 1.]);
println!("{}", u.dot(v));
// 2,0

let mut u = Vector::from([-1., 6.]);
let v = Vector::from([3., 2.]);
println!("{}", u.dot(v));
// 9,0
```

VII.1 Produit scalaire dans les espaces vectoriels complexes

Si les espaces vectoriels de nombres complexes vous intéressent et que vous maîtrisez déjà suffisamment les nombres complexes, vous pouvez vous renseigner sur les termes « **transposé conjugué** », « **algèbre sesquilinéaire** » et « **espace pré-Hilbert** ».



Les personnes intéressées par l'électronique, les systèmes de contrôle en ingénierie ou la mécanique quantique devraient *absolument* étudier les nombres complexes et l'algèbre sesquilinéaire.

Chapitre VIII

Exercice 04 - Norme

	Exercice : 04
Norm	
Fonctions mathématiques autorisées : `fonction « fused multiply-add », pow, max`	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : $O(n)$	

VIII.0.1 Objectif

Vous devez écrire des fonctions qui calculent différents types de normes.

Vous devez également fournir une fonction principale permettant de tester vos fonctions, prête à être compilée (si nécessaire) et exécutée.

VIII.0.2 Instructions

Soit v un vecteur d'un espace vectoriel V.

Vous devez implémenter les normes suivantes :

- Norme 1 : $\|v\|_1$ (également appelée norme « taxi » ou norme de Manhattan)

- Norme 2 : $\|v\|$ ou $\|v\|_2$ (également appelée norme euclidienne)

- Norme ∞ : $\|v\|_\infty$ (également appelée norme du suprême)

Les prototypes des fonctions à écrire sont les suivants :

```
impl<V> Vector::<V> {
    fn norm_1::<V>(&mut self) -> f32;
    fn norm::<V>(&mut self) -> f32;
    fn norm_inf::<V>(&mut self) -> f32;
}
```

Exemples :

```
let u = Vector::from([0., 0., 0.]);
println!("{}, {}, {}, {}", u.norm_1(), u.norm(), u.norm_inf());
// 0,0, 0,0, 0,0

let u = Vector::from([1., 2., 3.]);
println!("{}, {}, {}, {}", u.norm_1(), u.norm(), u.norm_inf());
// 6,0, 3,74165738, 3,0

let u = Vector::from([-1., -2.]);
println!("{}, {}, {}, {}", u.norm_1(), u.norm(), u.norm_inf());
// 3,0, 2,236067977, 2,0
```



Les normes renvoient toujours des nombres réels, même pour les vecteurs à valeurs complexes.

VIII.1 Inégalité triangulaire

C'est le moment idéal pour s'intéresser à ce qu'on appelle **l'inégalité triangulaire** pour les normes euclidiennes. Si les espaces vectoriels de dimension infinie (comme les espaces de suites ou de fonctions) vous intriguent, vous pouvez vous pencher sur **les normes de Hölder** et **l'inégalité de Minkowski**.

Vous pourriez également vous intéresser aux normes spécifiques aux espaces vectoriels de matrices, comme la **norme de Frobenius**.

Vous pourriez également vous intéresser aux « cousines » des normes, qui peuvent être négatives pour une coordonnée donnée. On les appelle **« pseudonormes »** et elles jouent un rôle important, par exemple, en relativité restreinte (voir aussi : *transformation de Lorentz*, *nombres complexes fractionnaires*, *géométrie hyperbolique*).

Remarque : les encyclopédies consacrées aux mathématiques, telles que Wikipédia, EncyclopediaOfMath et Wolfram MathWorld, peuvent s'avérer utiles de temps à autre, mais elles sont généralement plus adaptées à la révision d'un concept qu'à son apprentissage. Ne vous découragez donc pas si quelque chose ne vous semble pas clair d'emblée sur ces plateformes !

Chapitre IX Exercice

05 - Le cosinus

	Exercice : 05
Cosinus	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : $O(n)$	

IX.0.1 Objectif

Vous devez écrire des fonctions qui calculent le cosinus de l'angle formé par deux vecteurs donnés.

Vous devez également fournir une fonction principale permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.



`Vous devez utiliser les fonctions que vous avez écrites lors des deux exercices précédents.`

IX.0.2 Instructions

Soient $u, v \in \mathrm{K}^n$ des vecteurs.

Vous devez calculer $cos(u, v)$, le cosinus de l'angle entre les deux vecteurs u et v.



Rappel : l'utilisation de la fonction cos de la bibliothèque standard est bien sûr interdite.

Si l'un des vecteurs, ou les deux, est nul, le comportement est indéfini.

Si les vecteurs sont de tailles différentes, le comportement est indéfini.

Le prototype de la fonction à écrire est le suivant :

```
fn angle_cos::<K>(u: &Vector::<K>, v: &Vector::<K>) -> f32;
```

Exemples :

```
let u = Vector::from([1., 0.]);
let v = Vector::from([1., 0.]);
println!("{}", angle_cos(&u, &v));
// 1,0

let u = Vector::from([1., 0.]);
let v = Vector::from([0., 1.]);
println!("{}", angle_cos(&u, &v));
// 0,0

let u = Vector::from([-1., 1.]);
let v = Vector::from([ 1., -1.]);
println!("{}", angle_cos(&u, &v));
// -1,0

let u = Vector::from([2., 1.]);
let v = Vector::from([4., 2.]);
println!("{}", angle_cos(&u, &v));
// 1,0

let u = Vector::from([1., 2., 3.]);
let v = Vector::from([4., 5., 6.]);
println!("{}", angle_cos(&u, &v));
// 0,974631846
```

IX.1 Coefficient de corrélation de Pearson

Il existe de nombreuses analogies entre l'algèbre linéaire et les statistiques. Le « vecteur » en probabilités et en statistiques est généralement considéré comme une **variable aléatoire**. Une variable aléatoire est une fonction numérique. Son espace d'entrée est un ensemble de résultats possibles, comme $\Omega = \{1, 2, 3, 4, 5, 6\}$ pour les résultats qu'un dé peut donner, chaque résultat ayant une probabilité distincte (par exemple, un dé équitable attribuerait à chaque résultat possible une probabilité de $\frac{1}{6}$). Un tel espace d'entrée est appelé **espace de probabilité**. L'espace de sortie d'une variable aléatoire est un espace de valeurs, généralement représenté par R (par exemple, on gagne 3 € à chaque jet pair du dé et on perd 3 € à chaque jet impair).

Dans ce cadre, la **covariance** correspond au produit scalaire des espaces de variables aléatoires, la **variance** correspond à leur norme quadratique (une norme au carré, c'est-à-dire le produit scalaire d'un vecteur avec lui-même), et **l'écart-type** correspond à leur norme (la racine carrée d'une norme quadratique).

L'analogue de la cosinus pour les variables aléatoires s'appelle le **coefficient de corrélation de Pearson**. Ce coefficient est calculé à partir de deux variables aléatoires en utilisant la covariance comme produit scalaire, puis en normalisant, comme dans la formule habituelle du cosinus vectoriel.

- Si ce coefficient est proche de 1 (le cosinus est proche de 1, donc l'angle est petit, et les deux vecteurs sont presque colinéaires, avec la même orientation), les deux variables aléatoires sont fortement corrélées.

- Si ce coefficient est proche de 0 (le cosinus est proche de 0, donc l'angle est proche d'un angle droit, et les deux vecteurs sont presque perpendiculaires), les deux variables aléatoires ne sont pas corrélées.

- Si ce coefficient est proche de -1 (le cosinus est proche de -1, donc l'angle est grand (presque plat), et les deux vecteurs sont presque colinéaires, mais d'orientation opposée), les deux variables aléatoires sont fortement anticorrélées.

C'est l'un des nombreux exemples qui mettent en évidence la manière dont l'algèbre linéaire peut expliquer la géométrie sous-jacente aux statistiques.

Plus généralement, la science des données et les statistiques étudient des nuages de points de données, ce qui revient à « ne disposer que d'un ensemble d'échantillons des valeurs de sortie prises par une variable aléatoire, mais d'aucune information sur l'ensemble des événements/résultats (entrées) de cette variable aléatoire, ni sur leurs probabilités ». Dans ce cas très fréquent, on utilise plutôt un concept appelé « distribution de probabilité », qui permet de déduire des informations sur l'ensemble des résultats possibles Ω et leurs probabilités, sans connaître réellement cet espace, en se basant simplement sur la prévalence de certaines valeurs par rapport à d'autres. C'est ainsi que s'exerce le plus souvent la science des données, puisque nous disposons principalement d'observations, mais que nous souhaitons déterminer le modèle sous-jacent (une variable aléatoire cachée), qui devrait expliquer comment ces observations ont vu le jour.

Enfin, notons qu'il existe des variables aléatoires pouvant donner plusieurs valeurs de sortie (par exemple, si vous obtenez un 3 au dé, vous gagnez 2 €, mais perdez 5 barres chocolatées : il s'agit d'un vecteur à deux dimensions de valeurs réelles en sortie). Celles-ci peuvent être considérées de manière équivalente comme des vecteurs de variables aléatoires, et sont appelées **variables aléatoires multivariées**, ou **vecteurs aléatoires**. Elles revêtent une grande importance en science des données, car, en général, nous avons affaire à plusieurs

— par exemple, en examinant la répartition de la taille, du poids et du revenu (3 valeurs différentes) au sein d'une population d'individus.

Chapitre X

Exercice 06 - Produit vectoriel

	Exercice : 06
Produit vectoriel	
Fonctions mathématiques autorisées : `fonction « multiply-add » fusionnée`	
Complexité temporelle maximale : `N/A`	
Complexité spatiale maximale : `N/A`	

X.0.1 Objectif

Vous devez écrire une fonction qui calcule le produit vectoriel de deux vecteurs à trois dimensions.

Vous devez également fournir une fonction principale permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

X.0.2 Instructions

Soient $u, v \in V = \mathrm{R}^3$ des vecteurs.

Vous devez implémenter une fonction qui calcule le produit vectoriel $u \times v$.

Si l'un des vecteurs, ou les deux, n'est pas tridimensionnel, le comportement est

indéfini. Le prototype de la fonction à écrire est le suivant :

```
fn cross_product::<K>(u: &Vector::<K>, v: &Vector::<K>) -> Vector::<K>;
```

Exemples :

```
let u = Vector::from([0., 0., 1.]);
let v = Vector::from([1., 0., 0.]);
println!("{}", cross_product(&u, &v));
// [0.]
// [1.]
// [0.]

let u = Vector::from([1., 2., 3.]);
let v = Vector::from([4., 5., 6.]);
println!("{}", cross_product(&u, &v));
// [-3.]
// [6.]
// [-3.]

let u = Vector::from([4., 2., -3.]);
let v = Vector::from([-2., -5., 16.]);
println!("{}", cross_product(&u, &v));
// [17.]
// [-58.]
// [-16.]
```

X.1 Produit de Wedge et algèbre extérieure

Il existe un produit anticommutatif très important entre deux vecteurs, appelé **produit de Wedge** (ou **produit extérieur**), qui renvoie un objet appelé bivecteur. Alors qu'un vecteur (ou 1-vecteur) est un objet géométrique unidimensionnel (une droite ayant une certaine magnitude et une certaine orientation : représentée visuellement par une petite flèche de longueur et de sens déterminés), un bivecteur (ou 2-vecteur) est un objet géométrique bidimensionnel (un plan ayant une certaine magnitude et une certaine orientation : représenté visuellement par un petit parallélogramme de surface déterminée et de sens de rotation défini). Ce parallélogramme correspond précisément au parallélogramme formé par les deux vecteurs d'entrée.

Le produit extérieur s'étend aux bivexeurs et aux vecteurs, donnant des trivecteurs (vecteurs de dimension 3, qui correspondent à un volume orienté en 3D), etc. L'espace vectoriel de tous les vecteurs de dimension k jusqu'à une dimension maximale n est appelé **l'algèbre extérieure de dimension n sur un espace vectoriel** V et est noté $\Lambda^n(V)$. Ces algèbres extérieures revêtent une importance capitale dans les théories du calcul en dimensions supérieures et du calcul sur des espaces courbes (un domaine des mathématiques appelé **géométrie différentielle**). Si vous vous intéressez de près ou de loin à la physique avancée, vous serez probablement amené à les utiliser, que vous en ayez conscience ou non.

Le produit vectoriel est techniquement le dual (que l'on peut considérer comme le complément orthogonal, ici le vecteur perpendiculaire au parallélogramme) du produit de Wedge de deux vecteurs (bivecteur/parallélogramme). Le dual d'un k-vecteur dans un espace à n dimensions est toujours un $(n - k)$-vecteur. C'est pourquoi le produit vectoriel n'a de sens qu'en 3D (et

est donc une construction mathématique assez médiocre). Le produit en coin, en revanche, est toujours cohérent.

Notons que dans le contexte des **algèbres géométriques**, une sorte d'espace vectoriel dont les objets sont appelés « multivecteurs » et sont des sommes de k-vecteurs de rangs variés (différentes valeurs pour *k*), il existe une construction fondamentale appelée **produit géométrique**, qui est la somme du produit scalaire et du produit vectoriel. Les algèbres géométriques, bien que complexes à aborder au premier abord, offrent un cadre profond et magnifique qui va au-delà de l'algèbre linéaire et rend souvent les mathématiques avancées qui s'appuient sur l'algèbre linéaire beaucoup plus intuitives et naturelles. L'algèbre géométrique est, pour ainsi dire, « la meilleure façon » d'étendre les objets de l'algèbre linéaire et d'en faire un système amélioré.

Chapitre XI

Exercice 07 – Application linéaire, multiplication de matrices

	Exercice : 07
Application linéaire, multiplication de matrices	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : `voir ci-dessous`	
Complexité spatiale maximale : `voir ci-dessous`	

XI.0.1 Objectif

Vous devez écrire des fonctions qui multiplient une matrice par un vecteur ou une matrice par une matrice.

Vous devez également fournir une fonction principale permettant de tester vos fonctions, prête à être compilée (si nécessaire) et exécutée.

XI.0.2 Instructions

Soient $A \in \mathrm{R}^{m \times n}$, $B \in \mathrm{R}^{n \times p}$et $u \in \mathrm{R}^{(n),}$où $(m, n, p) \in \mathrm{N}^3$(représentés par des variables de type `u32`).

Vous devez implémenter des fonctions qui calculent :

- Au (qui renvoie un vecteur dans R^m) (complexité temporelle maximale $O(nm)$, complexité spatiale maximale $O(nm)$)

- *AB* (qui renvoie une matrice dans $\mathrm{R}^{m \times p}$) (complexité temporelle maximale $O(nmp)$, complexité spatiale maximale $O(nm + mp + np)$)

Les prototypes des fonctions à implémenter sont les suivants :

```
impl<K> Matrix::<K> {
    fn mul_vec::<K>(&mut self, vec : Vector::<K>) -> Vector::<K>
    ; fn mul_mat::<K>(&mut self, mat : Matrix::<K>) ->
    Matrix::<K> ;
}
```

Exemples :

```
let u = Matrix::from([
    [1., 0.],
    [0., 1.],
]);
let v = Vector::from([4., 2.]);
println!("{}", u.mul_vec(&v));
// [4.]
// [2.]

let u = Matrix::from([
    [2., 0.],
    [0., 2.],
]);
let v = Vector::from([4., 2.]);
println!("{}", u.mul_vec(&v));
// [8.]
// [4.]

let u = Matrix::from([
    [2., -2.],
    [-2., 2.],
]);
let v = Vector::from([4., 2.]);
println!("{}", u.mul_vec(&v));
// [4.]
// [-4.]



let u = Matrix::from([
    [1., 0.],
    [0., 1.],
]);
let v = Matrix::from([
    [1., 0.],
    [0., 1.],
```

```
]);
println!("{}", u.mul_mat(&v));
// [1., 0.]
// [0., 1.]

let u = Matrix::from([
    [1., 0.],
    [0., 1.],
]);
let v = Matrix::from([
    [2., 1.],
    [4., 2.],
]);
println!("{}", u.mul_mat(&v));
// [2., 1.]
// [4., 2.]

let u = Matrix::from([
    [3., -5.],
    [6., 8.],
]);
let v = Matrix::from([
    [2., 1.],
    [4., 2.],
]);
println!("{}", u.mul_mat(&v));
// [-14., -7.]
// [44., 22.]
```

XI.1 Matrices spéciales

Vous serez peut-être intéressé par les différents types de matrices carrées « spéciales » :

- les matrices de la forme λI_n, qui agissent comme une mise à l'échelle par la valeur λ
- les matrices diagonales, qui mettent à l'échelle chaque composante vectorielle indépendamment et sont faciles à élever à une puissance entière ;
- les matrices orthogonales, qui conservent les angles et les longueurs, et représentent une combinaison de rotations et de réflexions (symétriques bilatéralement, c'est-à-dire par miroir).

Il existe d'autres matrices spéciales, comme celles utilisées pour représenter les translations dans le lancer de rayons (modèle homogène), ou celles utilisées dans la diagonalisation ou la décomposition en valeurs singulières (SVD), mais les types de matrices cités ci-dessus sont les plus importants, car ce sont ceux que vous rencontrerez le plus souvent.

Elles forment également des groupes stables particuliers (au sens de « structure algébrique » du terme), appelés groupes de Lie, qui revêtent une grande importance en physique, car ils préservent une certaine grandeur mathématique (ce qu'on appelle une « symétrie » généralisée), ce qui se traduit par la conservation d'une grandeur physique (voir **le théorème de Noether**).

Chapitre XII

Exercice 08 - Trace

	Exercice : 08
Traite	
Fonctions mathématiques autorisées : `Aucune`	
Complexité temporelle maximale : $O(n)$	
Complexité spatiale maximale : `N/A`	

XII.0.1 Objectif

Vous devez écrire une fonction qui calcule la trace de la matrice donnée.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XII.0.2 Instructions

Soit $A \in K^{n \times n}$ une matrice carrée, où K est l'ensemble des nombres réels (représentés par le type `f32`) et $(m, n) \in N^2$ (représentés par deux variables de type `u32`).

La fonction doit calculer et renvoyer $Tr(A)$

Le prototype de la fonction à écrire est le suivant :

```
impl<K> Matrix::<K> {
    fn trace::<K>(&mut self) -> K;
}
```

Exemples :

```
soit u =
    Matrix::from([
    [1., 0.],
    [0., 1.],
]);
println!("{}", u.trace());
// 2,0

let u = Matrix::from([
    [2., -5., 0.],
    [4., 3., 7.],
    [-2., 3., 4.],
]);
println!("{}", u.trace());
// 9.0

let u = Matrix::from([
    [-2., -8., 4.],
    [1., -23., 4.],
    [0., 6., 4.],
]);
println!("{}", u.trace());
// -21,0
```

Chapitre XIII

Exercice 09 - Transposition

	Exercice : 09
Transposition	
Fonctions mathématiques autorisées : Aucune	
Complexité temporelle maximale : $O(nm)$	
Complexité spatiale maximale : $O(nm)$	

XIII.0.1 Objectif

Vous devez écrire une fonction qui calcule la matrice transposée d'une matrice donnée.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XIII.0.2 Instructions

Soit $A \in K^{m \times n}$, où K est l'ensemble des nombres réels (représentés par le type `f32`) et $(m, n) \in N^2$ (représentés par des valeurs de type `u32`).

La fonction doit renvoyer la matrice transposée $B \in K^{n \times m}$

Le prototype de la fonction à écrire est le suivant :

```
impl<K> Matrix::<K> {
    fn transpose::<K>(&mut self) -> Matrix::<K>;
}
```



La complexité temporelle est ici liée à l'affectation de valeurs, contrairement aux autres exercices où l'opération de base prise en compte est l'addition ou la multiplication.

XIII.1 Transposée conjuguée

Ici, il est utile de savoir que la transposition des matrices complexes utilise ce qu'on appelle la **transposée conjuguée**, qui repose sur l'opération de **conjugaison** des nombres complexes.

La raison de cette différence tient au fait que tout nombre complexe $z = a + ib$ peut être représenté de manière unique sous la forme d'une matrice réelle 2×2 de la forme :

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$$

appelée matrice de rotation-mise à l'échelle. L'addition, la multiplication ou l'inversion de ces matrices sont exactement équivalentes (« isomorphes ») aux mêmes opérations sur les nombres complexes ; tandis que le conjugué d'un nombre complexe est précisément la transposée de cette matrice réelle.

Si vous imaginez remplacer chaque coordonnée 1×1 d'une matrice complexe $n \times m$ par sa matrice de rotation-mise à l'échelle 2×2 correspondante, puis que vous effectuez une simple transposition sur cette matrice $2n \times 2m$, vous obtiendrez une matrice réelle $2m \times 2n$. Si vous inversez l'ordre des opérations et effectuez d'abord la transposition, avant de décomposer les coefficients en blocs 2×2, vous vous rendrez compte qu'il faut utiliser la transposée conjuguée, plutôt qu'une simple transposée, pour obtenir la même matrice finale $2m \times 2n$.

Il y a un diagramme commutatif caché ici. Saurez-vous le découvrir ? ;)

Chapitre XIV

Interlude 00 - Résolution de systèmes d'équations linéaires

Les systèmes d'équations linéaires sont des systèmes d'équations dans lesquels on ne trouve que **des combinaisons linéaires** de variables (une combinaison linéaire est une somme de variables pondérées, sans exposant sur les variables). Vous avez peut-être déjà utilisé des systèmes d'équations linéaires. En voici un exemple :

$$\begin{cases} 2x + 3y = 8 \\ -7x + 4y = 2 \end{cases}$$

Si vous maîtrisez désormais la multiplication matricielle, vous verrez facilement que vous pouvez représenter le système ci-dessus sous forme matricielle :

$$\begin{bmatrix} 2 & 3 \\ -7 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

On peut donc l'écrire algébriquement sous la forme :

$$Ab = c$$

On peut également l'écrire sous la forme d'une « matrice augmentée » :

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ -7 & 4 & 2 \end{array}\right]$$

Maintenant, pour résoudre le système, nous voulons calculer les coordonnées du vecteur inconnu *b* ; nous devons donc trouver un moyen de « le faire passer de l'autre côté du signe égal », comme nous l'avons fait pour les équations classiques à une seule variable. Comme la multiplication matricielle n'est pas *commutative* et qu'il existe des matrices qui réduisent les informations spatiales disponibles (ces matrices sont appelées

singulières et n'ont pas d'inverse), la division matricielle est particulière. Sous forme matricielle, on utilise **la matrice inverse** d'une matrice donnée pour effectuer la division. Mais les côtés par lesquels on multiplie l'inverse, et l'existence même de l'inverse d'une matrice donnée, sont deux questions très importantes qui méritent d'être examinées avec soin.

L'inverse de la matrice carrée A de taille $n \times n$, s'il existe, est noté A^{-1} et est défini comme suit :

$$A^{-1} A = AA^{-1} = I_n$$

où I_n est la matrice identité de R^n.

Ainsi, si elle existe, vous pouvez utiliser la matrice inverse pour résoudre le système (calculer les valeurs souhaitées du vecteur b contenant toutes les inconnues) :

$$\begin{aligned} Ab = c &\Leftrightarrow A^{-1} Ab = A^{-1} c && \text{(multiplication par l'inverse à gauche des deux côtés)} \\ &\Leftrightarrow I_n b = A^{-1} c && \text{(réduction des inverses à l'identité)} \\ &\Leftrightarrow b = A^{-1} c && \text{(solution calculable de l'équation)} \end{aligned}$$

Pour le système ci-dessus, la matrice inverse est :

$$A^{-1} = \begin{bmatrix} \frac{4}{29} & -\frac{3}{29} \\ \frac{7}{29} & \frac{2}{29} \end{bmatrix}$$

La solution est donc :

$$b = \begin{bmatrix} \frac{26}{29} \\ \frac{60}{29} \end{bmatrix}$$

De plus, le calcul de la matrice inverse n'est pas forcément facile, ni rapide. Mais vous vous en rendrez compte dans les exercices suivants. ;)

Chapitre XV

Exercice 10 – forme échelonnée par lignes

	Exercice : 10
forme échelonnée par lignes	
Fonctions mathématiques autorisées : Aucune	
Complexité temporelle maximale : $O(n^3)$	
Complexité spatiale maximale : $O(n^2)$	

XV.0.1 Objectif

Vous devez écrire une fonction qui calcule la forme échelonnée par lignes de la matrice donnée.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XV.0.2 Instructions

Soit $A \in K^{m \times n}$, où K est l'ensemble des nombres réels (représentés par le type `f32`) et $(m, n) \in N^2$(représentés par deux valeurs de type `u32`).

La fonction doit renvoyer la forme échelonnée par lignes de la matrice.



Les résultats peuvent être légèrement approximatifs (dans la mesure du raisonnable), car il est parfois difficile de rendre l'algorithme numériquement stable.

Le prototype de la fonction à écrire est le suivant :

```
impl<K> Matrix::<K> {
    fn row_echelon::<K>(&mut self) -> Matrix::<K>;
}
```

Exemples :

```
let u = Matrix::from([
    [1., 0., 0.],
    [0., 1., 0.],
    [0., 0., 1.],
]);
println!("{}", u.row_echelon());
// [1,0, 0,0]
// [0,0, 1,0, 0,0]
// [0,0, 1,0]

let u = Matrix::from([
    [1., 2.],
    [3., 4.],
]);
println!("{}", u.row_echelon());
// [1,0, 0,0]
// [0.0, 1.0]

let u = Matrix::from([
    [1., 2.],
    [2., 4.],
]);
println!("{}", u.row_echelon());
// [1,0, 2,0]
// [0.0, 0.0]

let u = Matrix::from([ [8.,
    5., -2., 4., 28.],
    [4., 2,5, 20., 4., -4.],
    [8., 5., 1., 4., 17.],
]);
println!("{}", u.row_echelon());
// [1,0 ; 0,625 ; 0,0 ; 0,0 ; -12,1666667]
// [0,0, 1,0, 0,0, -3,6666667]
// [0,0, 0,0, 1,0, 29,5]
```

Chapitre XVI

Exercice 11 - Déterminant

	Exercice : 11
Déterminant	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n^3)$	
Complexité spatiale maximale : $O(n^2)$	

XVI.0.1 Objectif

Vous devez écrire une fonction qui calcule le déterminant de la matrice donnée.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XVI.0.2 Instructions

Soit $A \in K^{n \times n}$ où K est l'ensemble des nombres réels (représentés par le type `f32`) et $n \in N$ et $n \leq 4$ (représenté par une valeur de type `u32`).

La fonction doit renvoyer le déterminant $det(A)$ de la matrice. Si la

matrice n'est pas carrée, le comportement est indéfini.

Étant donné que les algorithmes permettant de calculer le déterminant rapidement et avec précision pour des dimensions supérieures ont tendance à être assez complexes, nous avons limité les cas requis pour lesquels vous devez calculer le déterminant aux dimensions 4 et inférieures.



Pour les méthodes les plus simples, vous devrez utiliser une méthode spécifique par dimension, et peut-être « recycler » la technique issue des dimensions inférieures lorsqu'aucune autre ne semble disponible…



Vous devez être capable d'expliquer lors de l'évaluation :

- Que se passe-t-il lorsque $det(A) = 0$?
- Ce que représente géométriquement le déterminant dans l'espace vectoriel après avoir utilisé la matrice pour effectuer une transformation linéaire.
  Si vous ne le savez pas, c'est que vous n'avez pas regardé 3blue1brown, vilain élève.



Les résultats peuvent être un peu approximatifs (dans la mesure du raisonnable), car il peut être difficile de rendre l'algorithme numériquement stable.

Le prototype de la fonction à écrire est le suivant :

```
impl<K> Matrix::<K> {
    fn determinant::<K>(&mut self) -> K;
}
```

Exemples :

```
let u = Matrix::from([
    [ 1., -1.],
    [-1., 1.],
]);
println!("{}", u.determinant());
// 0,0

let u = Matrix::from([
    [2., 0., 0.],
    [0., 2., 0.],
    [0., 0., 2.],
]);
println!("{}", u.determinant());
// 8,0

let u = Matrix::from([
```

```
    [8., 5., -2.],
    [4., 7., 20.],
    [7., 6., 1.],
]);
println!("{}", u.determinant());
// -174,0

let u = Matrix::from([
    [ 8., 5., -2., 4.],
    [ 4., 2,5, 20., 4.],
    [ 8., 5., 1., 4.],
    [28., -4., 17., 1.],
]);
println!("{}", u.determinant());
// 1032
```

XVI.1 Le déterminant et l'algèbre extérieure

Si vous vous souvenez de la digression sur l'algèbre extérieure évoquée précédemment, vous voudrez peut-être savoir que les n-vecteurs au sein d'une algèbre extérieure de dimension n ont tendance à se comporter de manière très similaire à des scalaires, et sont appelés « pseudo-scalaires » pour cette raison.

Le déterminant est en fait la magnitude du pseudo-scalaire (la mesure du n-parallélépipède) obtenu par le produit en coin successif de tous les vecteurs colonnes d'une matrice carrée. Relisez cela lentement.

Cela devrait vous paraître logique si vous comprenez la nature géométrique du déterminant.

Chapitre XVII Exercice

12 - Inverse

	Exercice : 12
Inverse	
Fonctions mathématiques autorisées : `fonction « fused multiply-add »`	
Complexité temporelle maximale : $O(n^3)$	
Complexité spatiale maximale : $O(n^2)$	

XVII.0.1 Objectif

Vous devez écrire une fonction qui calcule la matrice inverse d'une matrice donnée.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XVII.0.2 Instructions

Soit $A \in \mathrm{K}^{n \times n}$ où K est l'ensemble des nombres réels (représentés par le type `f32`) et $n \in \mathrm{N}$ (représenté par une valeur de type `u32`).

La fonction doit renvoyer la matrice inverse $A^{-1} \in \mathrm{K}^{n \times n}$ telle que $A^{-1}A = I_n$, où I_n est la matrice identité.

Si la matrice est singulière, la fonction doit renvoyer une erreur. Si

la matrice n'est pas carrée, le comportement est indéfini.



Les résultats peuvent être quelque peu approximatifs (dans la mesure du raisonnable), car il peut être difficile de rendre l'algorithme numériquement stable. Plus précisément, le calcul de l'inverse est généralement numériquement instable lorsque la matrice est proche de la singularité ou lorsqu'elle est très grande.

Le prototype de la fonction à implémenter est le suivant :

```
impl<K> Matrix::<K> {
    fn inverse::<K>(&mut self) -> Result<Matrix::<K>>;
}
```

Exemples :

```
let u = Matrix::from([
    [1., 0., 0.],
    [0., 1., 0.],
    [0., 0., 1.],
]);
println!("{}", u.inverse());
// [1,0, 0,0]
// [0,0, 1,0, 0,0]
// [0,0, 1,0]

let u = Matrix::from([
    [2., 0., 0.],
    [0., 2., 0.],
    [0., 0., 2.],
]);
println!("{}", u.inverse());
// [0,5, 0,0, 0,0]
// [0,0, 0,5, 0,0]
// [0,0, 0,5]

let u = Matrix::from([
    [8., 5., -2.],
    [4., 7., 20.],
    [7., 6., 1.],
]);
println!("{}", u.inverse());
// [0,649425287, 0,097701149, -0,655172414]
// [-0,781609195, -0,126436782, 0,965517241]
// [0,143678161 ; 0,074712644 ; -0,206896552]
```

XVII.1 Pseudo-inverse de Moore-Penrose

Les matrices *de taille n* × *m* possèdent une « sorte » d'inverse, appelée **pseudo-inverse de Moore-Penrose**. Elle correspond à la meilleure approximation du « problème des moindres carrés ». Elle joue également un rôle dans la décomposition en valeurs singulières, qui est sans doute l'algorithme de décomposition matricielle le plus important (et il existe plus d'algorithmes de ce type que vous ne le pensez !).

Chapitre XVIII

Interlude 01 - Théorème de rang-nullité

Le théorème de rang-nullité énonce :

Soient E et F deux espaces vectoriels et soit $f \in L(E, F)$ une application linéaire. Alors :

$$rang(f) + nullité(f) = dim(im(f)) + dim(ker(f)) = dim(E)$$

Où,

- $rank(f)$ est le rang de la matrice de la transformation linéaire (dimension de l'image de f)
- $null(f)$ est la nullité de la matrice de la transformation linéaire (dimension du noyau de f)
- $dim(E)$ est la dimension de l'espace vectoriel (d'entrée) E

XVIII.0.1 Mais attendez, qu'est-ce qu'une image ? Qu'est-ce qu'un noyau ?

Soit f une fonction définie comme suit : $f : E \to F$ (qui associe des valeurs de l'ensemble E à l'ensemble F)

L'image de la fonction f, pour laquelle on a $im(f) \subseteq F$, est définie comme l'ensemble de toutes les valeurs $y \in F$ telles que :

$$\forall y \in im(f), \exists x \in E, f(x) = y$$

En termes simples, l'image d'une fonction est l'ensemble des points atteints par la fonction f dans l'espace de sortie.

Le **noyau** de la fonction f, pour lequel on a $ker(f) \subseteq E$, est défini comme l'ensemble de toutes les valeurs $x \in E$ telles que :

$$\forall x \in ker(f),\ f(x) = 0_F$$

En algèbre linéaire, le noyau est communément appelé « **espace nul** » et représente l'espace de tous les points qui sont « écrasés » à l'origine de l'espace vectoriel après la transformation linéaire.



Attention, « noyau » est l'un des termes les plus galvaudés en informatique et en mathématiques. Il a BEAUCOUP de significations différentes.

Chapitre XIX

Exercice 13 - Rang

	Exercice : 13
Rang	
Fonctions mathématiques autorisées : `Aucune`	
Complexité temporelle maximale : $O(n^3)$	
Complexité spatiale maximale : `N/A`	

XIX.0.1 Objectif

Vous devez écrire une fonction qui calcule le rang d'une matrice.

Vous devez également fournir une fonction main permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

XIX.0.2 Instructions

Soit $A \in K^{n \times m}$ où K est l'ensemble des nombres réels (représentés par le type `f32`) et $(m, n) \in N^2$ (représentés par deux valeurs de type `u32`).

La fonction doit renvoyer le rang de la matrice : $rank(A)$



Vous devez être capable d'expliquer lors de l'évaluation ce que représente le rang.

Le prototype de la fonction à écrire est le suivant :

```
impl<K> Matrix::<K> {
    fn rank::<K>(&mut self) -> usize;
}
```

Exemples :

```
let u = Matrix::from([
    [1., 0., 0.],
    [0., 1., 0.],
    [0., 0., 1.],
]);
println!("{}", u.rank());
// 3

let u = Matrix::from([
    [ 1., 2., 0., 0.],
    [ 2., 4., 0., 0.],
    [-1., 2., 1., 1.],
]);
println!("{}", u.rank());
// 2

let u = Matrix::from([
    [ 8., 5., -2.],
    [ 4., 7., 20.],
    [ 7., 6., 1.],
    [21., 18., 7.],
]);
println!("{}", u.rank());
// 3
```

Chapitre XX

Exercice 14 - Bonus : matrice de projection

	Exercice : 14
Bonus : Matrice de projection	
Fonctions mathématiques autorisées : `tan, fonction « fused multiply-add »`	
Complexité temporelle maximale : `N/A`	
Complexité spatiale maximale : `N/A`	

XX.0.1 Objectif

Vous devez écrire une fonction qui calcule une matrice de projection destinée au rendu d'objets 3D.

Vous devez également fournir une fonction principale permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

Un logiciel de visualisation est fourni avec le sujet. Il vous permettra de tester cet exercice.

XX.0.2 Instructions

La matrice de projection est un élément important des pipelines de rendu modernes. Dans OpenGL, par exemple, chaque point de l'espace 3D doit être converti en un point de l'espace 2D de l'écran.

Pour ce faire, nous utiliserons la formule suivante.

Soient $M, V, P \in \mathrm{R}^{4\times4}$ les matrices **de modèle**, **de vue** et **de projection** (il s'agit de certaines

parfois appelées respectivement matrices « **objet-monde** », « **monde-caméra** » et « **caméra-écran** ») et $s, p \in R^4$ la position du point à l'écran et la position du point dans l'espace 3D.

$$s = PV\ Mp$$

La matrice de vue V est définie comme suit :

$$V = RT$$

$$T = \begin{bmatrix} 1 & 0 & 0 & -p_x \\ 0 & 1 & 0 & -p_y \\ 0 & 0 & 1 & -p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Où :

- T est une matrice de translation qui place la caméra à l'origine de l'espace vectoriel.
- R est une matrice de rotation, qui fait pivoter l'espace vectoriel selon l'angle de la caméra.
- p est le vecteur représentant la position de la caméra
- (X, Y, Z) est la base de l'espace vectoriel après transformation.

La raison pour laquelle les matrices sont en 3 + 1 = 4 dimensions est que nous travaillons avec **des coordonnées homogènes** plutôt qu'avec **des coordonnées cartésiennes**. La linéarité implique que l'origine est conservée : cela signifie que les translations ne sont pas des applications linéaires. Les coordonnées homogènes constituent une astuce qui utilise une dimension supplémentaire avec l'espace K^{n+1} , afin de pouvoir utiliser les translations de manière à les rendre linéaires.

Dans cet exercice, nous ne construirons que la matrice de projection. Pour ce faire, la fonction prendra les arguments suivants :

- `fov` : champ de vision, l'angle du cône de vision
- `ratio` : le rapport de taille de la fenêtre : fenêtre de rendu $\frac{w}{h}$ où w et h sont respectivement la largeur et la hauteur de la
- `near` : la distance du plan proche.
- `far` : la distance du plan lointain.

Le prototype de la fonction à écrire est le suivant :

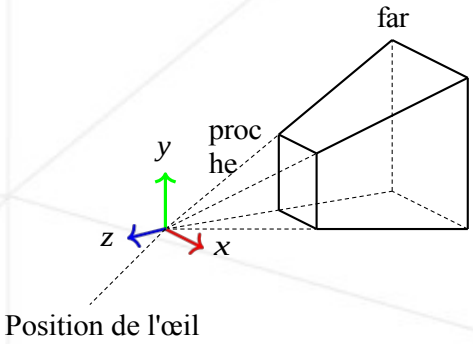


Figure XX.1 : Représentation graphique des explications ci-dessus. Tout ce qui doit être affiché à l'écran se trouve à l'intérieur du frustum.

```
fn projection(fov : f32, ratio : f32, near : f32, far : f32) -> Matrix::<f32> ;
```

Pour tester votre matrice de projection, vous pouvez l'intégrer dans le logiciel d'affichage comme suit : Exemple :

```
$ cat proj 1.,
0., 0., 0.
0., 1., 0., 0.
0., 0., 1., -1.
0., 0., 0., 0.
$ ./display
```

(L'exemple ci-dessus ne fonctionnera pas correctement pour une projection)



Le logiciel d'affichage s'attend à ce que la matrice d'entrée soit dans **l'ordre « column-major »**. Ses **coordonnées normalisées du périphérique** sont comprises dans les intervalles $[-1, 1]$ pour les axes X et Y, et $[0, 1]$ pour l'axe Z.



Une matrice de projection conçue pour des coordonnées normalisées de l'appareil (NDC) incorrectes et dans un ordre principal erroné peut sembler fonctionner à première vue, mais s'avérera défaillante après un examen minutieux. Essayez de jouer avec différents plans proche et lointain, et justifiez les changements que vous observez.



Figure XX.2 : Le résultat attendu

Chapitre XXI

Exercice 15 - Bonus : Espaces vectoriels complexes

	Exercice : 15
Bonus : Espaces vectoriels complexes	
Fonctions mathématiques autorisées : N/A	
Complexité temporelle maximale : N/A	
Complexité spatiale maximale : N/A	

XXI.0.1 Objectif

Vous avez désormais réussi à maîtriser toutes les notions fondamentales de l'algèbre linéaire.

Mais vous n'avez utilisé jusqu'à présent qu'un seul corps de scalaires : les nombres réels. Certes, c'est le plus utilisé, mais ce n'est certainement pas le seul.

Refaites tous les exercices précédents, en interprétant cette fois-ci K dans les signatures des fonctions non pas comme le corps R des nombres réels, mais comme le corps C des nombres complexes. Les espaces vectoriels sur les nombres complexes sont réputés difficiles à visualiser. Ils sont pourtant essentiels dans de nombreux domaines de l'électronique, de l'analyse des signaux et de la mécanique quantique, et ont même trouvé des applications dans l'apprentissage automatique (avec des tenseurs à valeurs complexes).

Vous devez également fournir une fonction principale permettant de tester votre fonction, prête à être compilée (si nécessaire) et exécutée.

Si vous vous sentez d'humeur audacieuse, vous pouvez vous pencher sur **les corps finis**, essayer de créer un type de corps fini et construire vos espaces vectoriels sur celui-ci. Ceux-ci sont plus faciles à visualiser que les espaces vectoriels sur les nombres complexes. Vous pouvez par exemple représenter n'importe quel espace vectoriel de

de dimension 2, basé sur un corps fini, comme un espace de carreaux à la surface d'un beignet ! :D



Cet exercice bonus explique pourquoi nous avons conservé la plupart de nos déclarations avec un K générique, plutôt que de remplacer K par f32
par défaut dans les déclarations. Cela vise à mettre en avant un aspect puissant de l'algèbre abstraite : la généricité. Grâce à un concept appelé « foncteur », on peut souvent assembler des structures algébriques comme on le ferait avec des briques Lego pour construire des espaces mathématiques plus intéressants, tout comme les types génériques fonctionnent en informatique !

Chapitre XXII

Remise et évaluation par les pairs

Déposez votre travail dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué lors de la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.